

Final Project for Supervised Machine Learning

Customer Churn Prediction

```
In [237... import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [238... df = pd.read_csv("E:/Data Solution 360/Supervised Machine Learning/Final Assignment/synthetic_customer_
df
```

```
Out[238...      CustomerID  Age  Gender  Tenure  MonthlyCharges  Contract  PaymentMethod  TotalCharges  Churn
0              1   56  Female     68           147.58  Two year    Bank transfer    10052.03    No
1              2   69   Male     32           22.54  Month-
to-month    Mailed check     686.78    No
2              3   46  Female     10           52.47  One year    Electronic check   537.88    No
3              4   32   Male     22          109.67  Month-
to-month    Mailed check   2390.04    Yes
4              5   60  Female     54          130.98  Month-
to-month    Credit card    7081.28    No
...           ...   ...   ...     ...           ...      ...           ...           ...
99995          99996   31   Male     49           26.07  Month-
to-month    Electronic check   1220.50    No
99996          99997   64  Female     44          123.22  Month-
to-month    Mailed check    5384.38    No
99997          99998   48  Other     32           75.37  Month-
to-month    Credit card    2372.33    Yes
99998          99999   42  Female     60          114.00  Month-
to-month    Mailed check    6826.55    No
99999         100000   72   Male     57           66.76  One year    Credit card     3828.60    No
```

100000 rows × 9 columns

```
In [239... df.shape
```

```
Out[239... (100000, 9)
```

```
In [240... df.isnull().sum()
```

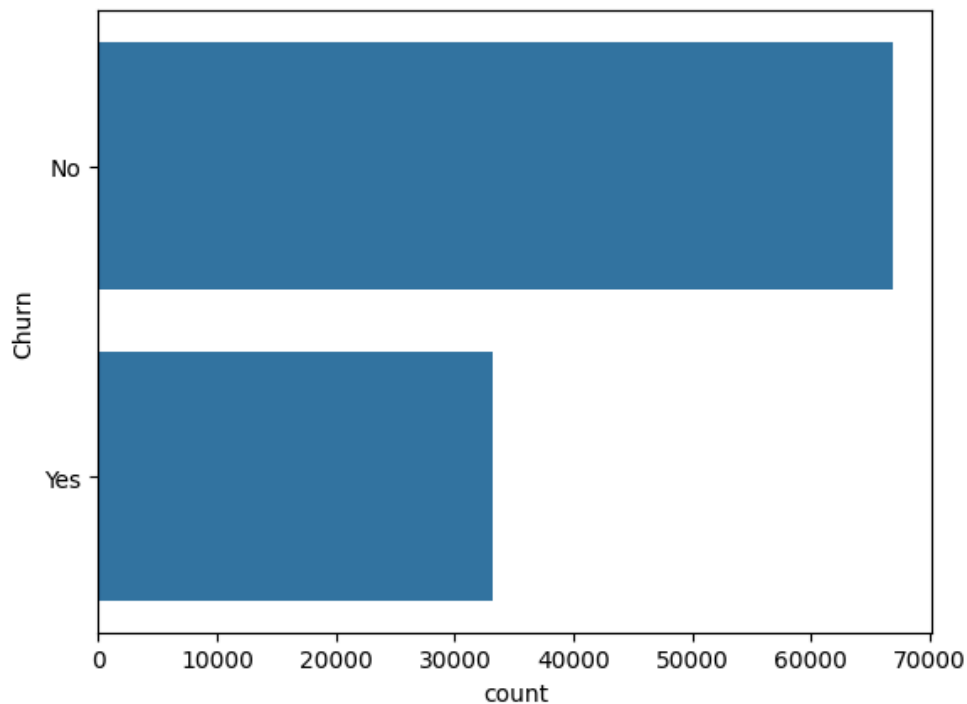
```
Out[240...] CustomerID      0
           Age          0
           Gender       0
           Tenure       0
           MonthlyCharges 0
           Contract     0
           PaymentMethod 0
           TotalCharges 0
           Churn        0
           dtype: int64
```

```
In [241...] df.duplicated().sum()
```

```
Out[241...] np.int64(0)
```

```
In [242...] sns.countplot(df['Churn'])
```

```
Out[242...] <Axes: xlabel='count', ylabel='Churn'>
```



```
In [243...] df['Churn'].value_counts()
```

```
Out[243...] Churn
No      66856
Yes     33144
Name: count, dtype: int64
```

```
In [244...] churned = df[df.Churn == 'Yes'].shape[0]
           retained = df[df.Churn == 'No'].shape[0]
```

```
In [245...] # Customer Left the company
ch_per = ((churned/(churned + retained)) * 100)
ch_per
```

```
Out[245...] 33.144
```

```
In [246...] # Customer Stayed the company
re_per = ((retained/(churned + retained))*100)
re_per
```

Out[246... 66.85600000000001

In [247... df.describe()

Out[247...

	CustomerID	Age	Tenure	MonthlyCharges	TotalCharges
count	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000
mean	50000.500000	49.027240	36.527090	79.974933	2926.140256
std	28867.657797	18.177862	20.789552	40.491961	2388.163672
min	1.000000	18.000000	1.000000	10.000000	-118.430000
25%	25000.750000	33.000000	18.000000	44.720000	963.667500
50%	50000.500000	49.000000	37.000000	80.000000	2268.060000
75%	75000.250000	65.000000	54.000000	115.050000	4394.327500
max	100000.000000	80.000000	72.000000	150.000000	10831.460000

Visualization

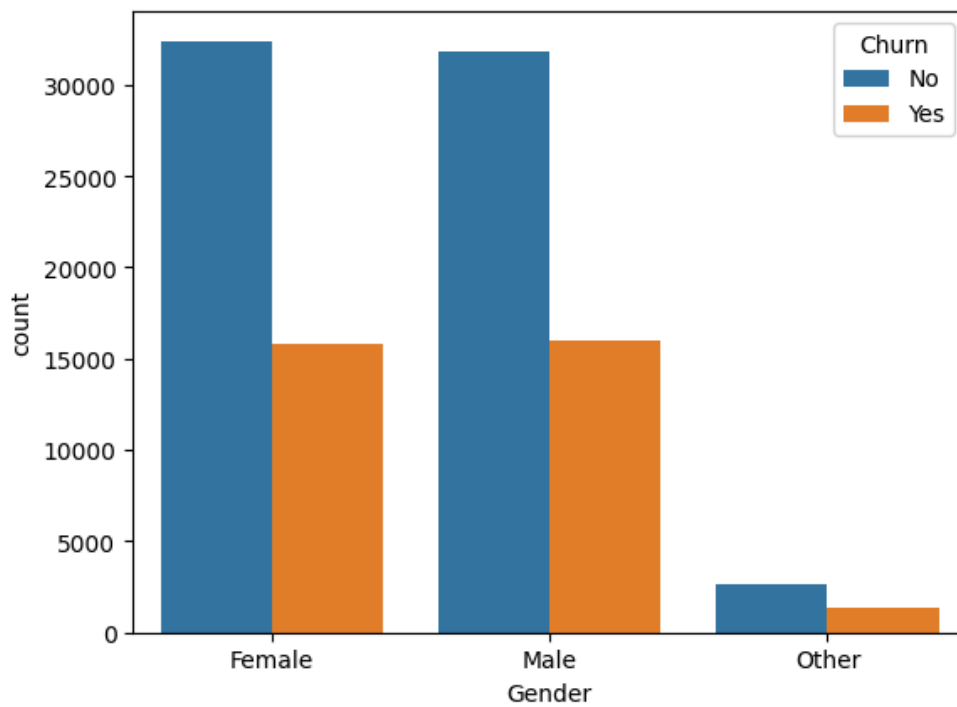
In [248... df.head(4)

Out[248...

	CustomerID	Age	Gender	Tenure	MonthlyCharges	Contract	PaymentMethod	TotalCharges	Churn
0	1	56	Female	68	147.58	Two year	Bank transfer	10052.03	No
1	2	69	Male	32	22.54	Month-to-month	Mailed check	686.78	No
2	3	46	Female	10	52.47	One year	Electronic check	537.88	No
3	4	32	Male	22	109.67	Month-to-month	Mailed check	2390.04	Yes

In [249... sns.countplot(x='Gender', hue='Churn', data=df)

Out[249... <Axes: xlabel='Gender', ylabel='count'>

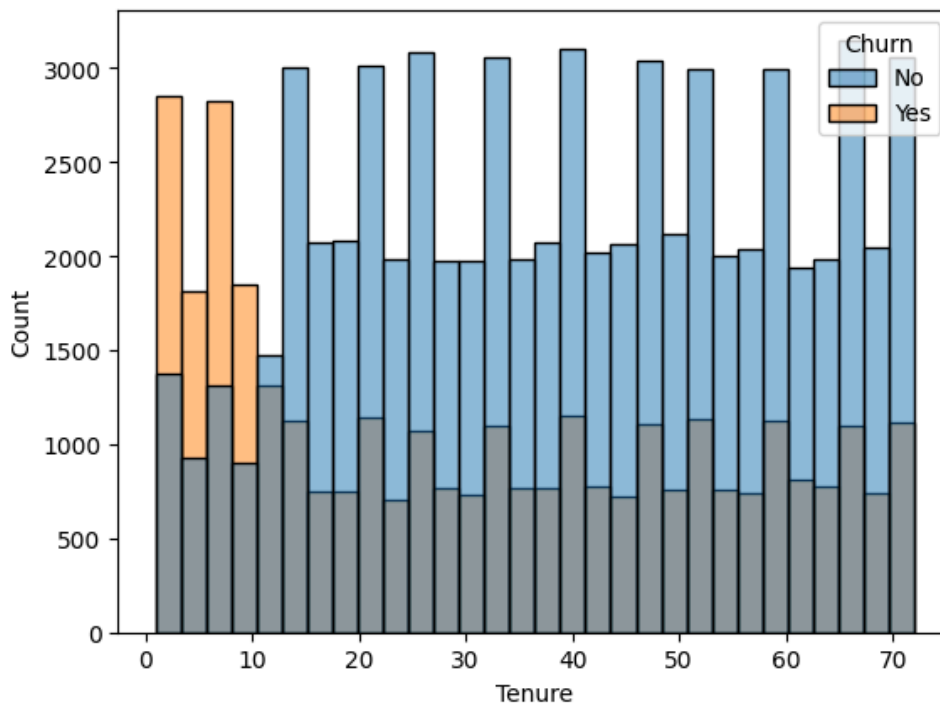


```
In [250...] df['Gender'].value_counts()
```

```
Out[250...] Gender
Female      48256
Male       47787
Other        3957
Name: count, dtype: int64
```

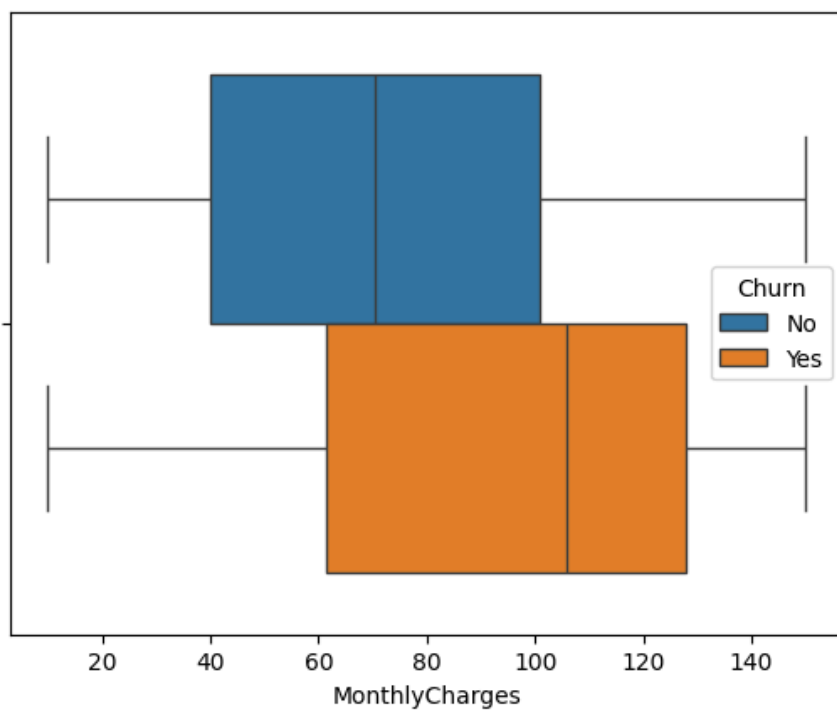
```
In [251...] sns.histplot(data=df, x="Tenure", hue="Churn", bins=30)
```

```
Out[251...] <Axes: xlabel='Tenure', ylabel='Count'>
```



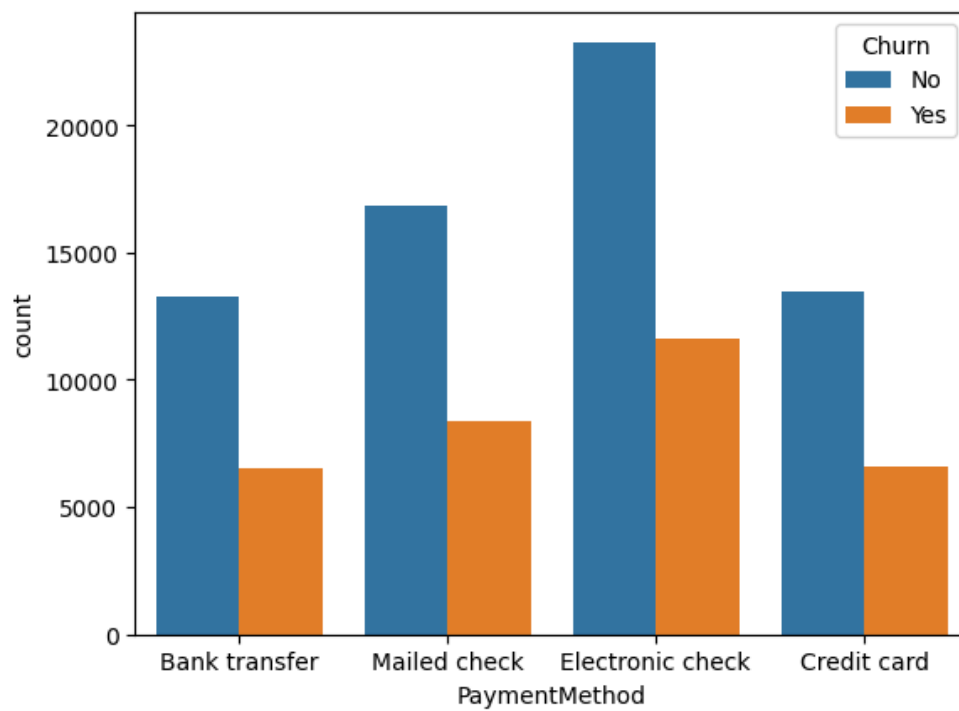
```
In [252...] sns.boxplot(x="MonthlyCharges",hue="Churn" ,data=df)
```

Out[252... <Axes: xlabel='MonthlyCharges'>



In [253... `sns.countplot(x='PaymentMethod', hue='Churn', data=df)`

Out[253... <Axes: xlabel='PaymentMethod', ylabel='count'>



In [254... `df.head(5)`

Out[254...

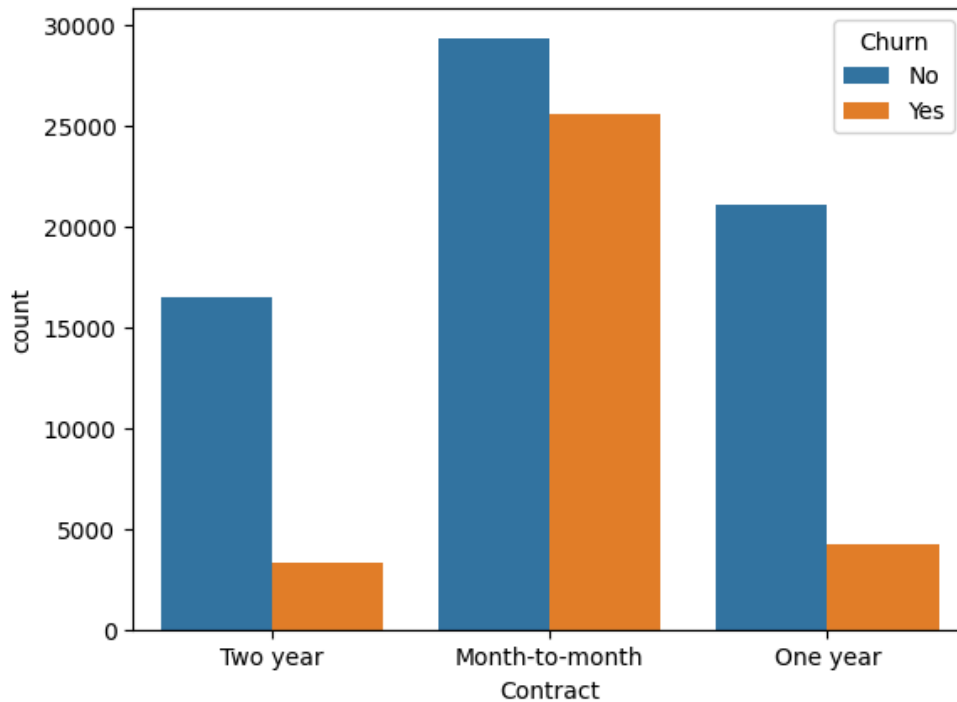
	CustomerID	Age	Gender	Tenure	MonthlyCharges	Contract	PaymentMethod	TotalCharges	Churn
0	1	56	Female	68	147.58	Two year	Bank transfer	10052.03	No
1	2	69	Male	32	22.54	Month-to-month	Mailed check	686.78	No
2	3	46	Female	10	52.47	One year	Electronic check	537.88	No
3	4	32	Male	22	109.67	Month-to-month	Mailed check	2390.04	Yes
4	5	60	Female	54	130.98	Month-to-month	Credit card	7081.28	No

In [255...

```
sns.countplot(x='Contract', hue='Churn', data=df)
```

Out[255...

```
<Axes: xlabel='Contract', ylabel='count'>
```



Data Preprocessing

In [256...

```
# Drop Customer ID and Age
data = df.drop(['CustomerID', 'Age'], axis=1)
```

In [257...

```
data.head(5)
```

Out[257...

	Gender	Tenure	MonthlyCharges	Contract	PaymentMethod	TotalCharges	Churn
0	Female	68	147.58	Two year	Bank transfer	10052.03	No
1	Male	32	22.54	Month-to-month	Mailed check	686.78	No
2	Female	10	52.47	One year	Electronic check	537.88	No
3	Male	22	109.67	Month-to-month	Mailed check	2390.04	Yes
4	Female	54	130.98	Month-to-month	Credit card	7081.28	No

```
In [258... data.shape
```

```
Out[258... (100000, 7)
```

Label Encoding

```
In [259... from sklearn.preprocessing import LabelEncoder
```

```
In [260... for column in data.columns:  
    if data[column].dtype == 'object':  
        data[column] = LabelEncoder().fit_transform(data[column])
```

```
In [261... data
```

```
Out[261...      Gender  Tenure  MonthlyCharges  Contract  PaymentMethod  TotalCharges  Churn  
0         0      68         147.58         2             0        10052.03      0  
1         1      32         22.54         0             3         686.78      0  
2         0      10         52.47         1             2         537.88      0  
3         1      22        109.67         0             3        2390.04      1  
4         0      54        130.98         0             1        7081.28      0  
...      ...      ...           ...           ...           ...           ...      ...  
99995      1      49         26.07         0             2         1220.50      0  
99996      0      44        123.22         0             3         5384.38      0  
99997      2      32         75.37         0             1         2372.33      1  
99998      0      60        114.00         0             3         6826.55      0  
99999      1      57         66.76         1             1         3828.60      0
```

100000 rows × 7 columns

Train and Test Data

```
In [262... x = data.drop('Churn', axis=1)  
y = data['Churn']
```

```
In [263... y
```

```
Out[263... 0      0  
1      0  
2      0  
3      1  
4      0  
..  
99995  0  
99996  0  
99997  1  
99998  0  
99999  0  
Name: Churn, Length: 100000, dtype: int64
```

Scaling the data

```
In [264... from sklearn.preprocessing import StandardScaler
```

```
In [265... feature_x = StandardScaler().fit_transform(x)
```

```
In [266... feature_x
```

```
Out[266... array([[ -0.9757253 ,  1.51388865,  1.66960066,  1.70980422, -1.55837885,
          2.98385134],
        [  0.77599424, -0.21775902, -1.41843515, -0.82153276,  1.26683556,
        -0.93769597],
        [ -0.9757253 , -1.27598816, -0.67927239,  0.44413573,  0.32509742,
        -1.00004544],
        ...,
        [  2.52771379, -0.21775902, -0.1137252 , -0.82153276, -0.61664072,
        -0.23189911],
        [ -0.9757253 ,  1.12907805,  0.8402961 , -0.82153276,  1.26683556,
          1.63323364],
        [  0.77599424,  0.98477408, -0.32636106,  0.44413573, -0.61664072,
          0.37789045]], shape=(100000, 6))
```

Split Data

```
In [267... from sklearn.model_selection import train_test_split
```

```
In [268... xtrain, xtest , ytrain, ytest = train_test_split(feature_x,y, test_size=0.25, random_state=42)
```

```
In [269... xtest
```

```
Out[269... array([[ -0.9757253 , -0.1696577 , -1.42658495, -0.82153276, -0.61664072,
          -0.9067306 ],
        [  0.77599424, -0.362063 ,  1.38880278,  0.44413573, -0.61664072,
          0.41751105],
        [  0.77599424,  1.03287541,  1.11961219, -0.82153276,  1.26683556,
          1.82867348],
        ...,
        [ -0.9757253 , -0.93927889,  0.12360704,  0.44413573,  0.32509742,
          -0.64109003],
        [ -0.9757253 , -0.26586035, -1.5621681 , -0.82153276, -1.55837885,
          -0.99021357],
        [ -0.9757253 , -1.46839345, -1.63749207,  0.44413573, -0.61664072,
          -1.17868568]], shape=(25000, 6))
```

Logistic Regression Model

```
In [270... from sklearn.linear_model import LogisticRegression
```

```
In [271... model = LogisticRegression()
```

```
In [272... model.fit(xtrain,ytrain)
```

```
Out[272... ▾ LogisticRegression ⓘ ⓘ
  ▸ Parameters
```

```
In [273... pred = model.predict(xtest)
```

```
In [274... pred.shape
```


Out[274... (25000,)

Performance Metics

```
In [275... from sklearn.metrics import accuracy_score
```

```
In [276... # accuracy score
accuracy_score(ytest,pred)
```

Out[276... 0.72344

```
In [277... # confusion metrics
from sklearn.metrics import confusion_matrix
```

```
In [278... confusion_matrix(ytest,pred)
```

Out[278... array([[14329, 2343],
 [4571, 3757]])

```
In [279... # classification report
from sklearn.metrics import classification_report
print(classification_report(ytest,pred))
```

	precision	recall	f1-score	support
0	0.76	0.86	0.81	16672
1	0.62	0.45	0.52	8328
accuracy			0.72	25000
macro avg	0.69	0.66	0.66	25000
weighted avg	0.71	0.72	0.71	25000

Decission Tree

```
In [280... from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier()
dt.fit(xtrain,ytrain)

pred_dt = dt.predict(xtest)

accuracy_score(ytest,pred_dt)
```

Out[280... 0.68104

Random Forest

```
In [281... from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier()

rf.fit(xtrain,ytrain)

pred_rf = rf.predict(xtest)

accuracy_score(ytest,pred_rf)
```

Out[281... 0.72108

KNN

```
In [282... from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier()

knn.fit(xtrain,ytrain)

pred_knn = knn.predict(xtest)

accuracy_score(ytest,pred_knn)
```

Out[282... 0.71352

Naive Base

```
In [283... from sklearn.naive_bayes import GaussianNB

nb = GaussianNB()

nb.fit(xtrain,ytrain)

pred_nb = nb.predict(xtest)

accuracy_score(ytest,pred_nb)
```

Out[283... 0.7336

Final View

```
In [284... models = pd.DataFrame({
    'Model':['Logistic Regression','Decision Tree','Random Forest','KNN','Naive Bayes'],
    'Accuracy':[
        accuracy_score(ytest,pred),
        accuracy_score(ytest,pred_dt),
        accuracy_score(ytest,pred_rf),
        accuracy_score(ytest,pred_knn),
        accuracy_score(ytest,pred_nb)
    ]
})

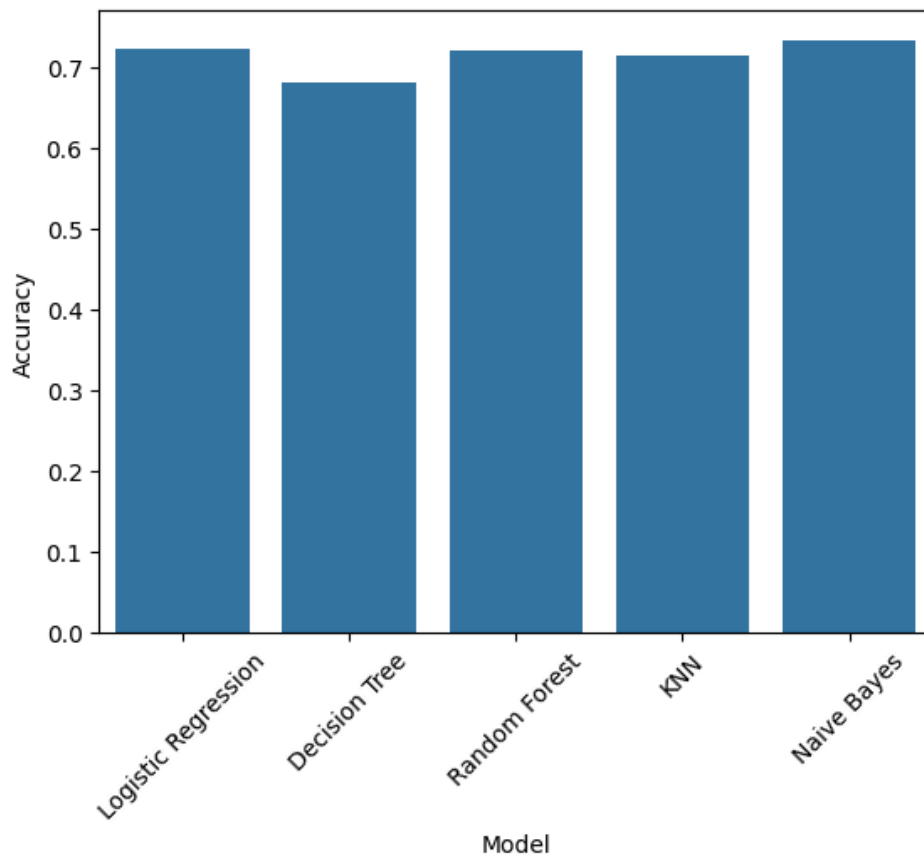
models
```

Out[284...

	Model	Accuracy
0	Logistic Regression	0.72344
1	Decision Tree	0.68104
2	Random Forest	0.72108
3	KNN	0.71352
4	Naive Bayes	0.73360

Accuracy Graph:

```
In [285... sns.barplot(x='Model', y='Accuracy', data=models)
plt.xticks(rotation=45)
plt.show()
```



Conclusion:

In this dataset, Naive Bayes achieved the highest accuracy (73.36%).

Logistic Regression, Random Forest, and KNN produced almost similar results.

The performance of Decision Tree was lower compared to the other models.

Feature importance analysis from Random Forest shows which features have the most impact on churn prediction.

With hyperparameter tuning, Random Forest could potentially achieve even better performance in the future.